The ACFT11 Flight Control System

A Structured Description — Architecture, Control Cascade, and Decision Logic

Model: ACFT11_a_121.slx · ECEF 6-DOF full-mission simulation
Supersonic two-stage carrier UAV (~28.5 t), envelope M0.4–M4
Controller: INDI inner loop · ACAH / TECS / γ-law / L1 outer loops · 17-phase sequencer
Generated 2026-10-26

1 Design philosophy

A single control law cannot fly this vehicle. The ACFT11 carrier accelerates from a standing start on a runway to beyond Mach 2, zoom-climbs to roughly 27 km on rocket power, coasts through air too thin for its control surfaces, glides home, and lands on an ILS-style approach. Each of those regimes has different dynamics, different effectors, and different goals.

The design answer, used throughout aerospace practice, is a **layered, nested architecture** governed by a **supervisory sequencer**:

> *The sequencer decides **what** the aircraft should be doing; the guidance layer decides **where** to go to do it; the control laws decide **how** to make the airframe go there; and one robust inner loop makes the surfaces produce the motion the laws ask for — regardless of flight condition.*

Two principles make this work:

- **Separation of guidance and control.** Guidance is kinematics ("fly this flight path"); control is dynamics ("produce this moment"). Keeping them apart lets each guidance law be a simple geometric or energy rule while one inner loop absorbs all the nonlinear airframe behaviour.
- **Feedback linearisation of the fast dynamics (INDI).** The inner loop uses *measured* angular acceleration to cancel the airframe's own dynamics, so the outer loops see a clean, almost flight-condition-independent plant and can be tuned with constant classical gains across the whole M0.4–M4 envelope.

2 Functional decomposition

At the top level the FCS is six cooperating functions wrapped around the plant. Figure 1 shows the functional flow: references and sensor data enter on the left, physical motion emerges on the right, and measurements are fed back.

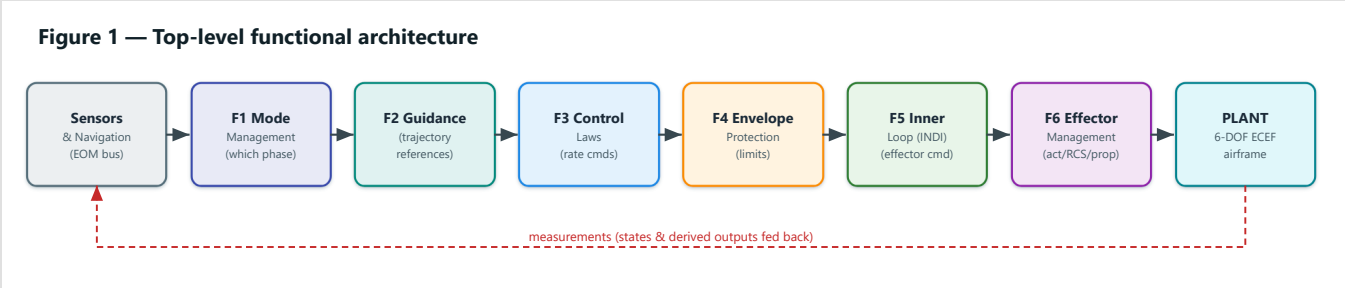


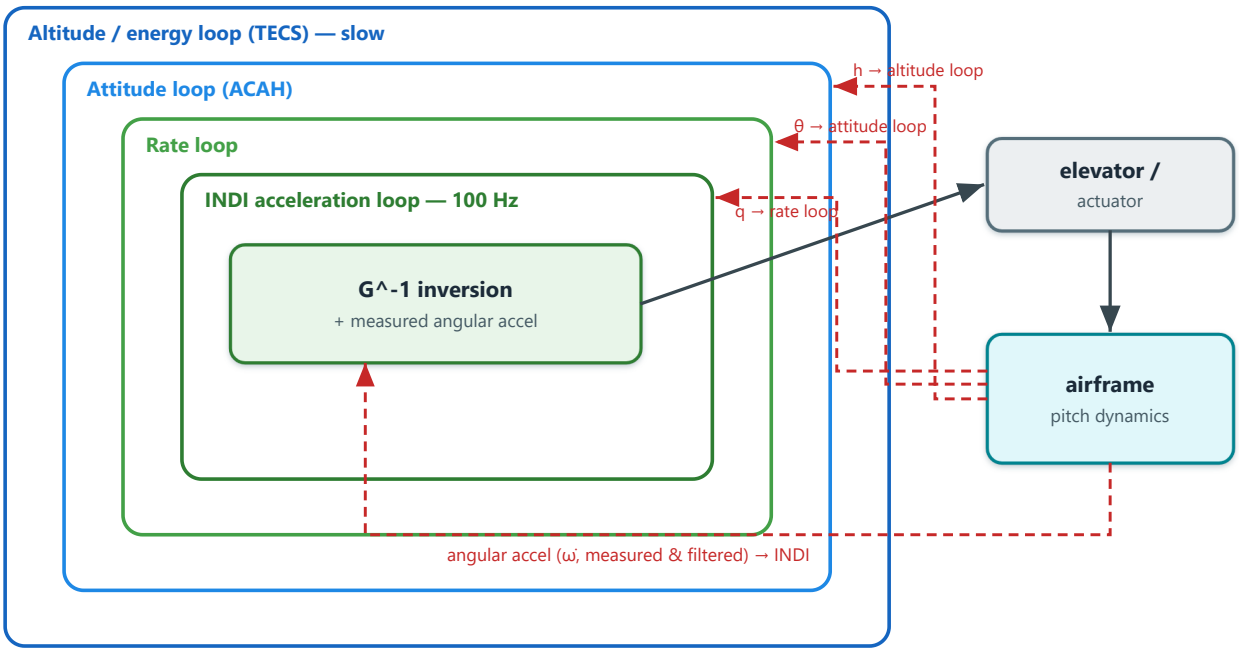
Figure 1 — Top-level functional architecture of the flight control system.

Function	Responsibility	Realisation in the model
F1 Mode Management	choose mission phase; publish guidance mode, active law, propulsion mode	`MissionManager` (Stateflow FSM)
F2 Guidance	convert phase into γ, V, h, ψ references	guidance functions, `GlideCompute`, `L1law`
F3 Control Laws	references → body-rate commands + throttle	TECS, ACAH, γ-law
F4 Envelope Protection	clamp commands to α, Nz, bank limits	α/Nz/bank limiters, `NzLimiter`
F5 Inner Loop	rate error → surface increments via G^{-1}	`INDI_Inner`
F6 Effector Management	drive actuators / RCS / engines	actuators, `RCS_ctrl`, Propulsion Mgr + `SpoolLag`

3 The control cascade

Each function that touches the fast dynamics is itself a **loop**, and the loops are **nested** — outer loops are slow and set the targets for the faster loops inside them. This is the classical structure of every modern autopilot. Figure 2 shows the four nested longitudinal loops.

Figure 2 — Nested control loops (pitch axis; roll/yaw identical in form)



Each loop sets the target for the faster loop nested inside it; feedback (dashed) closes each one. Outer = slow, inner = 100 Hz.

Figure 2 — Nested control loops (pitch axis shown; roll and yaw are identical in form).

Reading from the inside out:

- **INDI acceleration loop (100 Hz).** Given a desired angular acceleration `v = K·(rate_cmd − rate)`, it commands the surface increment $\delta = \delta_0 + G^{-1}\cdot(v - \dot{\omega}_{measured})$. Because it subtracts the measured acceleration, it cancels the airframe's own moment and forces $\dot{\omega} \approx v$. The effectiveness `G` is scheduled from the aircraft's own aero tables and live dynamic pressure, so the inversion always matches the plant.
- **Rate loop.** `v = Kq·(q_cmd − q)` — sets how quickly the body rate tracks its command. With the inner loop linearised, this closes as a first-order lag $q/q_{cmd} \approx Kq/(s+Kq)$.
- **Attitude loop (ACAH).** `q_cmd = Kθ·(θ_cmd − θ)` — turns an attitude error into a rate command; closes as a well-damped second order.
- **Altitude / energy loop (TECS).** The outermost loop turns an altitude / energy error into the attitude command and, in parallel, a throttle command.

The lateral axis has the identical shape: **track → bank (L1) → roll-rate → INDI**. The yaw axis adds a washed-out yaw damper for Dutch-roll.

> *Why the nesting order matters: each loop only needs to be faster than the one outside it, not faster than physics. Stability is designed one loop at a time, from the inside out — an auditable procedure.*

4 Mode logic — the mission decision flow

Function F1 is a finite-state machine. Figure 3 is its decision flowchart: each phase runs its guidance until a guard condition (a sensor threshold from `fcs_params.m`) is satisfied, then it transitions. This is the heart of the "what should the aircraft be doing now" logic. When the owning law changes at a transition, the handover is designed to be **bumpless** — the newly active law is initialised to the current command so nothing steps.

Figure 3 — Mission sequencer: phase-transition decision flow

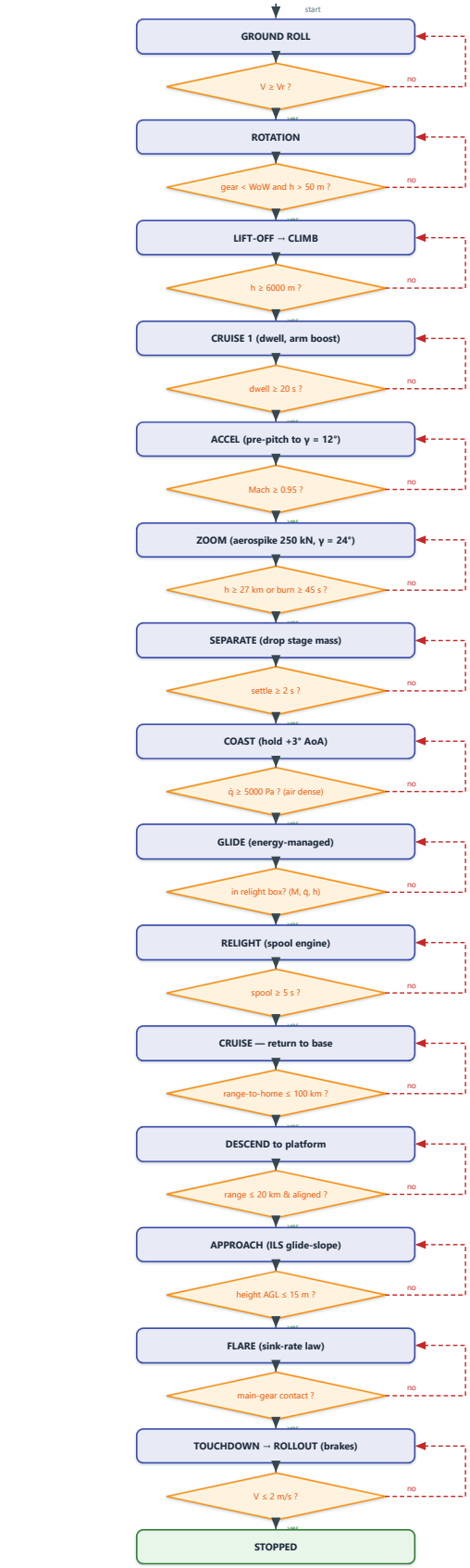


Figure 3 — Mission sequencer phase-transition decision flow. Each diamond is a guard; the vehicle loops in place until it is satisfied.

5 Within-step control decision flow

Every 100 Hz tick the FCS must decide which law owns the pitch/throttle command and which effectors to use. Figure 4 is that per-step decision flow — it turns the sequencer's `law_select` and the current dynamic pressure into actual surface and thruster commands.

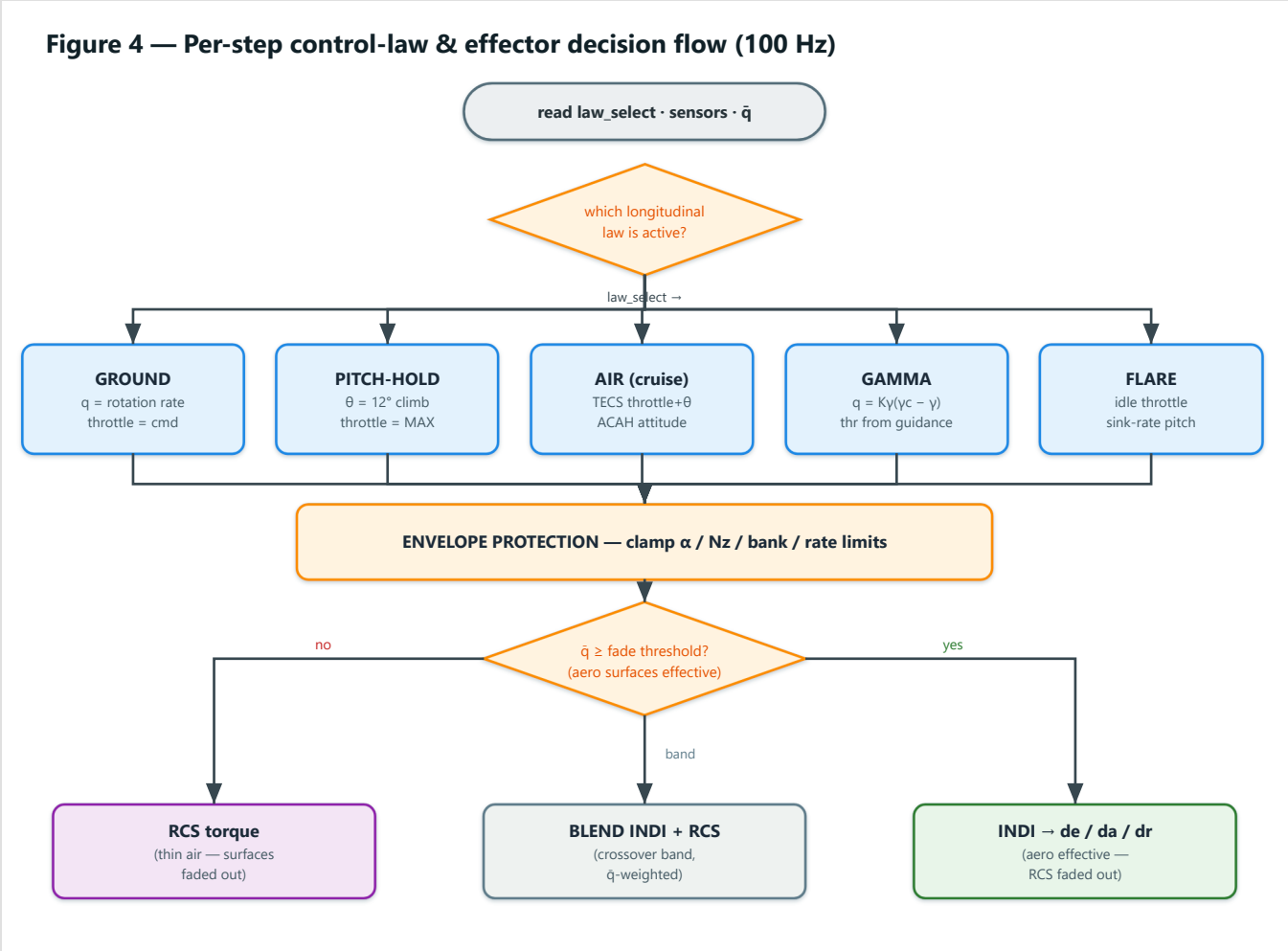


Figure 4 — Per-step control-law and effector decision flow (runs every 100 Hz step).

The single most important variable in this decision is **dynamic pressure (q̄)**: it fades the aerodynamic surface authority *in* as speed builds and fades the reaction-control authority *out*, with a deliberate overlap so attitude control is never lost through the thin-air coast.

6 Requirements → design rationale

A systems-engineering description closes the loop by tracing each design feature back to the requirement or problem it answers.

#	Requirement / problem	Design feature	§
R1	Fly ground-roll to Mach 2+ to landing	17-phase supervisory sequencer	4
R2	One controller across M0.4–M4	INDI inner loop + scheduled G	3
R3	Decouple speed & altitude	TECS energy/balance split	3
R4	Keep Nz < 2.5 g at boost pull-up	ACCEL pre-pitch + γ-rate limit + Nz limiter	4,5
R5	Controllable attitude in vacuum	q̄-blended RCS, +3° AoA hold	5
R6	No elevator slam at rocket cutoff	COAST flight-path tracking + 1 s thrust decay	4
R7	No surface flutter / respect rate limits	per-step rate clamps inside INDI	3
R8	Realistic thrust build-up	first-order spool lag (τ = 2 s)	5
R9	Stay inside the flight envelope always	α / Nz / bank protection layer	5
R10	Smooth law hand-overs	bumpless transfer at each transition	4

Companion documents: FCS_CONTROLLER_THEORY.txt (full theory), FCS_SDOMAIN_BLOCK_DIAGRAM.txt (classic s-domain loops), EQUATIONS_OF_MOTION_THEORY.txt and EOM_OUTPUT_STATES_DERIVATION.txt (the plant). All gains, limits and thresholds in the figures are the values in fcs_params.m; the INDI effectiveness schedule is built by setup_gschedule.m.